

Aim: To find the Minimum Spanning Tree (MST) of a graph, which is the set of edges that connects all vertices with the minimum possible total edge weight, without forming any cycles by using Kruskal's Algorithm.

Description:

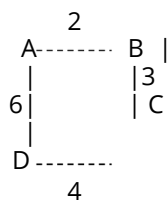
Kruskal's Algorithm is a greedy approach to find a graph's Minimum Spanning Tree (MST). It aims to connect all vertices with the lowest possible total edge weight, avoiding any loops. The algorithm begins by listing all edges and their weights. It then sorts these edges from cheapest to most expensive. Starting with an empty MST, it considers each edge in sorted order. If adding an edge doesn't create a cycle, it's included in the MST. This continues until all vertices are part of the MST.

Algorithm:

1. List Edges: Create a list of all edges in the graph along with their weights.
2. Sort Edges: Sort this list of edges in non-decreasing order of their weights (from smallest weight to largest weight).
3. Initialize MST: Create an empty set to store the edges of the Minimum Spanning Tree (MST).
4. Iterate Through Sorted Edges: Go through the sorted list of edges. For each edge (u,v):
 - o Check if adding this edge to the MST would create a cycle.
 - o If adding the edge does not create a cycle, add it to the MST.
5. Termination: Continue until all vertices are connected by the edges in the MST.

Example:

Consider the following weighted undirected graph:



Steps of Kruskal's Algorithm:

1. Edges with weights: (A, B): 2, (B, C): 3, (D, C): 4, (A, D): 6
2. Sorted Edges: (A, B): 2, (B, C): 3, (D, C): 4, (A, D): 6
3. Initialize MST: $MST = \{\}$
4. Iterate:
 - o Edge (A, B) weight 2: Adding it doesn't create a cycle. $MST = \{(A, B)\}$
 - o Edge (B, C) weight 3: Adding it doesn't create a cycle. $MST = \{(A, B), (B, C)\}$
 - o Edge (D, C) weight 4: Adding it doesn't create a cycle. $MST = \{(A, B), (B, C), (D, C)\}$
 - o Edge (A, D) weight 6: Adding it would create a cycle (A-B-C-D-A). Skip this edge.
5. Resulting MST: $\{(A, B), (B, C), (D, C)\}$ with a total weight of $2+3+4=9$.

Code:

```
def sort_edges(edges):
```

```
    for i in range(len(edges)):
```

```
        for j in range(i + 1, len(edges)):
```

```
            if edges[i][2] > edges[j][2]:
```

```
                edges[i], edges[j] = edges[j], edges[i]
```

```
    return edges
```

```
def find_parent(parent, v):
```

```
    while parent[v] != v:
```

```
        v = parent[v]
```

```
    return v
```

```
def union(parent, u, v):
```

```
    u_root = find_parent(parent, u)
```

```
    v_root = find_parent(parent, v)
```

```
    parent[u_root] = v_root
```

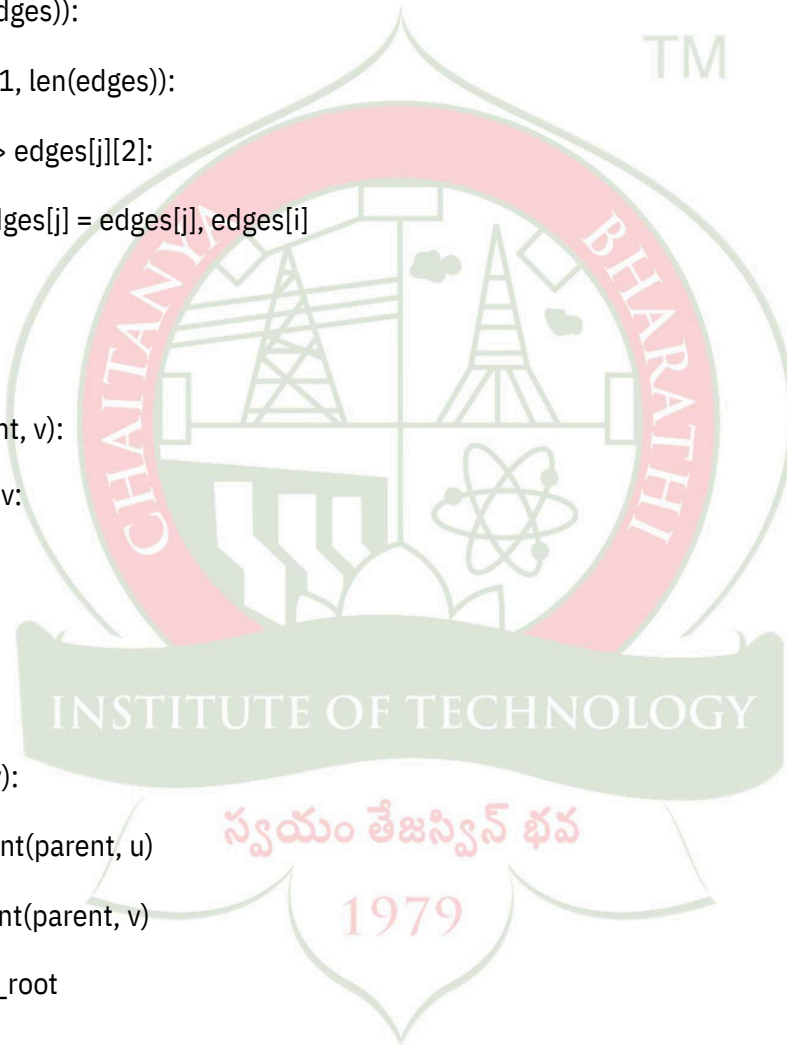
```
def kruskal(n, edges):
```

```
    parent = [i for i in range(n)]
```

```
    mst = [None] * (n - 1)
```

```
    cost = 0
```

```
    edges = sort_edges(edges)
```



```
mst_index = 0

for i in range(len(edges)):

    u, v, w = edges[i]

    u_parent = find_parent(parent, u)

    v_parent = find_parent(parent, v)

    if u_parent != v_parent:

        mst[mst_index] = (u, v, w)

        mst_index += 1

        cost += w

        union(parent, u, v)

print("\nMinimum Spanning Tree using Kruskal's Algorithm:")

for u, v, w in mst:

    if u is not None:

        print(f"{u} - {v} : {w}")

print("Total cost:", cost)
```

```
n = int(input("Enter number of vertices: "))

edges = []

m = int(input("Enter number of edges: "))

for _ in range(m):

    u, v, w = map(int, input("Enter edge (u v weight): ").split())

    edges[len(edges):len(edges)+1] = [[u, v, w]]

kruskal(n, edges)
```

Output

Enter number of vertices: 4 Enter number of edges: 5

Enter edge (u v weight): 0 1 10 Enter edge (u v

weight): 0 2 6 Enter edge (u v weight): 0 3 5 Enter

edge (u v weight): 1 3 15 Enter edge (u v weight): 2 3

4 Minimum Spanning Tree using Kruskal's Algorithm:

2 - 3 : 4 0 - 3 : 5 0 - 1 : 10 Total cost: 19

Time Complexity:

- ☐ Sorting the edges takes $O(E \log E)$, where E is the number of edges.
- ☐ Checking for cycles can be done efficiently, and its contribution is less significant than the sorting time for a general graph.
- ☐ Overall time complexity: $O(E \log E)$.

Space Complexity:

- ☐ The algorithm requires space to store the graph edges and the resulting MST.
- ☐ Space complexity: $O(E+V)$, where V is the number of vertices.

Aim: To find the Minimum Spanning Tree (MST) of a weighted, undirected graph, connecting all vertices with the minimum total edge weight and without forming any cycles by using Prim's Algorithm.

Description:

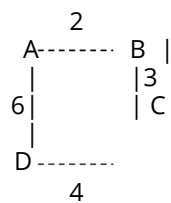
Prim's Algorithm constructs the MST step by step, starting from a single vertex. It maintains a set of vertices already in the MST and another set of remaining vertices. In each iteration, it selects the lightest edge that connects a vertex within the MST to a vertex outside of it. This edge and the new vertex are then added to the MST. The process continues until all vertices are included in the MST, effectively growing the tree by adding the cheapest reachable vertices.

Algorithm:

1. Initialize MST: Create an empty set to store the edges of the MST.
2. Select Start Vertex: Choose an arbitrary vertex to begin the MST.
3. Maintain Neighboring Edges: Keep track of the edges that connect a vertex in the MST to a vertex not yet in the MST.
4. Select Minimum Edge: Find the edge with the smallest weight among the tracked edges.
5. Add to MST: Add the selected edge and its corresponding new vertex to the MST.
6. Update Neighbors: Update the tracked neighboring edges based on the newly added vertex.
7. Repeat: Repeat steps 4-6 until all vertices are included in the MST.

Example:

Consider the graph:



Starting with vertex A:

1. MST: {}, Included: {A}
2. Edges to explore: (A, B): 2, (A, D): 6
3. Choose (A, B): 2. MST: {(A, B)}, Included: {A, B}
4. Edges to explore: (A, D): 6, (B, C): 3
5. Choose (B, C): 3. MST: {(A, B), (B, C)}, Included: {A, B, C}
6. Edges to explore: (A, D): 6, (C, D): 4
7. Choose (C, D): 4. MST: {(A, B), (B, C), (C, D)}, Included: {A, B, C, D}

Code:

```
def prim(n, graph):
```

```
    selected = [0] * n
```

```
selected[0] = 1

cost = 0

edges = [None] * (n - 1)

mst_index = 0

for _ in range(n - 1):
    min_edge = [0, 0, 999999]
    for i in range(n):
        if selected[i] == 1:
            for j in range(n):
                if selected[j] == 0 and graph[i][j] != 0:
                    if graph[i][j] < min_edge[2]:
                        min_edge = [i, j, graph[i][j]]
    u, v, w = min_edge
    selected[v] = 1
    edges[mst_index] = (u, v, w)
    mst_index += 1
    cost += w

print("\nMinimum Spanning Tree using Prim's Algorithm:")

for u, v, w in edges:
    if u is not None:
        print(f"{u} - {v} : {w}")

print("Total cost:", cost)
```



```
n = int(input("Enter number of vertices: "))
```

```
graph = [[0] * n for _ in range(n)]
```

```
for i in range(n):
```

```
    graph[i] = list(map(int, input(f"Enter row {i} of the adjacency matrix: ").split()))
```

```
prim(n, graph)
```

Output

Enter number of vertices: 4 Enter row 0 of the

adjacency matrix: 0 10 6 5 Enter row 1 of the

adjacency matrix: 10 0 0 15 Enter row 2 of the

adjacency matrix: 6 0 0 4 Enter row 3 of the

adjacency matrix: 5 15 4 0 Minimum Spanning

Tree using Prim's Algorithm: 0 - 3 : 5 3 - 2 : 4 0 - 1

: 10 Total cost: 19

Time Complexity:

- ☐ $O(V^2)$ using an adjacency matrix.
- ☐ $O(E \log V)$ using a binary heap and adjacency list.

Space Complexity:

- ☐ $O(V)$ to keep track of included vertices and potentially edge weights.
- ☐ $O(E)$ if using an adjacency list.
- ☐ Overall: $O(V+E)$.